

NTC_CC105 – DATA STRUCTURE W/LAB

WEEK 5 LABORATORY INSTRUCTION

Laboratory Activity 5

Instructor	Prof. Darwin C. Vargas, MIT
Program	BSIT
Academic Year	2026–2027
Programming Language	C++

1. Student Name	Quintela, Nicole M. / Serdenia Rihanna Shealya
2. Student Number	425001249 / 425002643
3. Section	2.4 BSIT
4. Date	Oct 5, 2026
5. Laboratory Activity No.	Laboratory Activity 5
6. Laboratory Title	Linear Architecture Synthesis & Peer Code Audit

GUIDE QUESTIONS

Answer the following questions in your final report:

1. Data Structure Selection & Trade-Offs: Why are dynamic arrays preferred over fixed-size static arrays for unpredictable batch workloads, and what hidden computational costs occur during memory reallocations?

- A dynamic array grows with your data, so you never have to guess the size in advance. A fixed-size array either wastes memory or runs out of space. The hidden cost is reallocation. In my code, when `size == capacity`, `resize()` creates a new array twice as big, copies every element with a loop, and frees the old one. That copy is $O(n)$, but doubling the capacity keeps the average cost of adding an item small.

2. Memory Hygiene & Lifecycle: Explain the role of the C++ destructor (`~DynamicArray()`) in managing heap allocation safety. What system failure occurs if `delete[]` is omitted in long-running C++ applications?

- The destructor `~DynamicArray()` calls `delete[] data`, which returns the heap memory automatically when the object goes out of scope. Without it, the memory is never given back (a memory leak). In long-running applications, leaks pile up until the system slows down or crashes.

3. Defensive Programming & Bounds Enforcement: Contrast the behavior of raw array indexing (`array[i]`) with bounds-checked operations. Why is missing upper-bound validation (`idx >= size`) dangerous in security-critical codebases?

- Raw indexing (`array[i]`) doesn't verify the position, so a wrong index silently reads or writes memory it shouldn't. My `removeAt()` checks `index < 0 || index >= size` and rejects invalid input with a message. Missing the upper-bound check is dangerous in security-critical code because attackers can exploit it to leak data or take control of the program.

4. Algorithmic Overhead & Code Smells: How did eliminating redundant nested loops inside the array deletion routine change the runtime performance of the system?

- The old deletion routine made extra, unnecessary passes through the data, so it got slow as the list grew ($O(n^2)$). The refactored `removeAt()` uses a single loop to shift elements left, so deletion is now $O(n)$ and noticeably faster on large data.

5. Value of Peer Code Audits: Why are structured peer code reviews essential in commercial software development prior to merging feature branches into production repositories?

- A second reviewer catches bugs, leaks, and security gaps the author missed. Reviews also keep the code consistent and easier to maintain. Fixing problems before merging costs far less than fixing them in production.

REFLECTION QUESTIONS

Include thoughtful responses to the following reflection prompts in your submission:

1. Experiential Reflection: What was the most challenging defect to locate inside the legacy codebase, and how did systematic unit testing assist you in pinpointing the root cause compared to random debugging?

- The hardest issues were the silent ones, like the missing bounds check and the memory leak, because the program appeared to work fine. Unit tests let me test one function at a time with specific cases (empty array, last index, invalid index), so I could see exactly where it failed. This was much more reliable than random debugging, which relies on guesswork.

2. Architectural Evaluation: Knowing what you now know about dynamic array reallocations and shift-based element deletions, what linear architecture alternative (e.g., Linked Lists, Queues, Stacks) would you recommend for systems that require frequent mid-list insertions without reallocating buffers?

- For frequent insertions in the middle of a list, I would recommend a linked list. Inserting only requires updating a couple of links, so nothing needs to be shifted and no buffer needs to be reallocated. The downside is slower access by position and slightly more memory per item.

3. Professional Growth: How has evaluating another programmer's code during the Peer Code Review altered your approach to writing clean, well-documented C++ code for your upcoming Milestone 1 project?

- Reviewing someone else's code showed me how much unclear names, missing comments, and unchecked inputs slow down the reader. For Milestone 1, I will use clear names, add short comments, validate inputs, and always match `new[]` with `delete[]`. I will also test edge cases before submitting.